AI Engineer Journey

Day 3 Session Summary

Student	Vaibhav Nandurkar
Date	Thursday, May 28, 2026
Phase	Phase 1 — Python Foundations
Sessions	Morning + Noon + Evening + Night
Projects	Pandas Cleaning · Sales Dashboard · SQL Fundamentals · Student DB CLI
GitHub	github.com/VaibhavNandurkar/ai-engineer-journey

■■ Morning Session — Pandas Data Cleaning

10:00 – 11:30 · 6 checkpoints completed

What was covered

Method	What it does
pd.DataFrame(dict)	Build a table from a Python dictionary — columns = keys, values = lists
df.isnull()	Returns True/False grid showing which cells are missing
df.isnull().sum()	Counts missing values per column
df.dropna()	Drops any row that has at least one missing value
df.fillna(value)	Fills missing cells with a given value (mean, string, etc.)
df.drop_duplicates()	Removes rows where all column values are identical
df.to_csv() / read_csv()	Write DataFrame to CSV file / read CSV back into DataFrame

Full pipeline pattern

Save dirty data → read back fresh → fillna marks with mean → fillna name with 'Unknown' → drop_duplicates → save clean CSV. Always use df.copy() before modifying to preserve the original.

■■ Noon Session — Sales Dashboard Analyser

13:00 – 14:30 · 7 checkpoints completed · groupby + agg

What was built

A full sales analytics script processing 18 rows of product/month/region data. Computes revenue, monthly totals, top product, region breakdown, and prints a formatted dashboard.

Method / Pattern	What it does
df[col] = df[a] * df[b]	Create revenue column by multiplying two columns element-wise
groupby(col)[col].sum()	Total a numeric column grouped by a category
.reindex(list)	Force a custom sort order on a Series index (e.g. Jan→Jun)
.sort_values(ascending=False)	Sort a Series highest-first
.idxmax()	Return the index label of the row with the highest value
.agg({col: func})	Apply multiple aggregations across multiple columns in one call
series[series > series.mean()]	Boolean filter on a Series — keeps only rows above threshold

■ Evening Session — SQL Fundamentals

17:00 – 18:30 · 6 checkpoints completed · SQLite, CRUD, JOIN

SQL / Python	What it does
sqlite3.connect(file)	Connect to (or create) a .db file — no install needed
conn.cursor()	Create cursor object — used to run all SQL commands
CREATE TABLE IF NOT EXISTS	Define schema; IF NOT EXISTS prevents error on re-run
INSERT OR IGNORE INTO ... VALUES (?,?,?)	Insert row using ? placeholders — never use f-strings in SQL
cursor.executemany(sql, list)	Run same INSERT for every tuple in a list
SELECT * WHERE col > val	Filter rows by condition
ORDER BY col ASC/DESC	Sort query results
INNER JOIN ON t1.fk = t2.pk	Combine two tables on a matching key column
UPDATE SET col=? WHERE id=?	Modify existing row — always use ? placeholders
DELETE FROM WHERE id=?	Remove a row by condition
cursor.rowcount	How many rows were affected by last UPDATE or DELETE
conn.commit() / conn.close()	Save changes to disk / close the connection

■ Night Session — Student Database CLI

21:00 – 22:30 · 7 checkpoints · OOP + SQLite full CRUD app

What was built

A fully OOP CLI app where StudentDB class wraps all SQLite operations. Menu-driven while True loop calls 6 methods: add, view, search, update marks, delete, stats. Input validated with try/except ValueError. rowcount used to detect missing IDs.

Class / Method	What it does
StudentDB.__init__(db_name)	Connect to SQLite, create cursor, call _create_table()
_create_table()	CREATE TABLE with AUTOINCREMENT id — private method
add_student(name,age,course,marks)	INSERT 4 columns — id auto-assigned by SQLite
view_all()	SELECT * + fetchall() loop — prints formatted rows or empty message
search_student(name)	SELECT WHERE name LIKE ? with %name% for partial match
update_marks(id, marks)	UPDATE SET marks WHERE id — rowcount check for missing id
delete_student(id)	DELETE WHERE id — rowcount check for missing id
show_stats()	Single SELECT with COUNT, AVG, MAX, MIN — fetchone() unpack
main()	while True menu — try/except for bad input, conn.close() on exit

■ Key Concepts Reinforced Today

Concept	Where used
Method chaining	groupby().sum().reindex() — multiple operations in one line
? placeholders in SQL	All INSERT/UPDATE/DELETE — never use f-strings in SQL
AUTOINCREMENT	Night project — SQLite assigns id automatically
cursor.rowcount	Detect if UPDATE/DELETE found a matching row
Trailing comma tuple	(value,) not (value) — required for single-value SQL params
df.copy()	Preserve original DataFrame before modifying
OOP wrapping I/O	StudentDB encapsulates all DB logic — clean separation

■■ Your Roadmap

Phase	Topics	Timeline	Status
Phase 1	Python OOP, Git, File I/O, Pandas, SQL	May 26 – Jun 14	In Progress
Phase 2	Scikit-learn, ML, Deep Learning	Jun 15 – Jul 19	Upcoming
Phase 3	LLM APIs, RAG, Agents	Jul 20 – Sep 6	Upcoming
Phase 4	MLOps, Deploy, Job Apps	Sep 7 – Dec 31	Upcoming

■ Tomorrow — Day 4

Slot	Time	Topic
Morning	10:00 – 11:30	SQL Advanced — GROUP BY, HAVING, subqueries, indexes
Noon	13:00 – 14:30	Pandas + SQL combined — read SQL into DataFrame, analyse, write back
Evening	17:00 – 18:30	Matplotlib basics — line, bar, pie charts
Night	21:00 – 22:30	Full Visualisation Dashboard — sales data → 4 charts in one script

---

To start Day 4, tell Claude: **Day 4 — upload this PDF, FCC Dark Style, ai-engineer-journey**

GitHub: github.com/VaibhavNandurkar/ai-engineer-journey · Goal: AI Engineer by Jan 2027